Team Number: 17

Team Members:

Drew Franke
Vrishank Kulkarni
Alexander Lanter
Davina Love
Noah Mast
Mo Osby
Kayaan Patel

Project Name: Rogue Swordsman ? (WIP)

Project Synopsis: Rogue Swordsman is a turn-based fantasy roguelike where players fight through a procedurally generated dungeon, collecting loot and relics, to defeat a final boss.

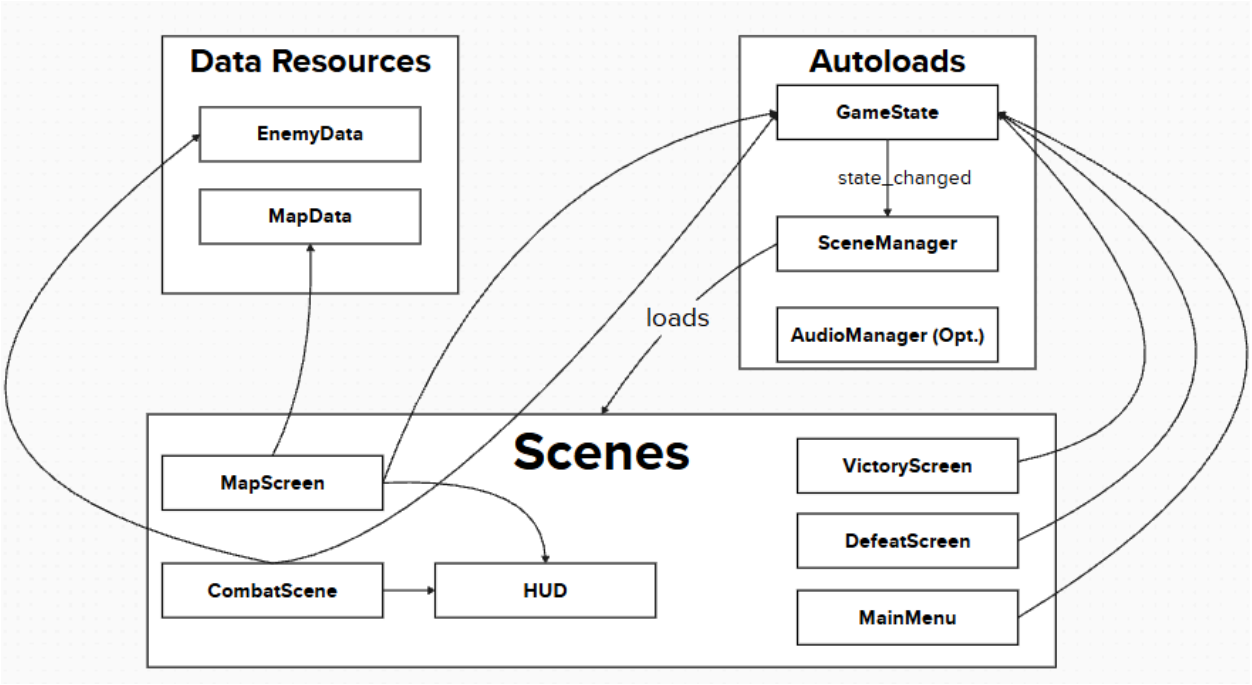
Architecture Description: Next Page.

Architecture Description Outline:

Overview

Rogue Swordsman is a turn-based roguelike built in Godot 4 with GDScript and exported to HTML5 so it can be played in a web browser. Our architecture separates the game into three main concerns: map navigation, combat, and run state. Godot scenes will be used to handle presentation and behavior, custom resources holding data such as enemy stats, autoload singletons keep run wide state that must survive scene changes, and signals let scenes communicate without referencing each other directly. The three diagrams below describe the system: a component diagram for static structure, a class diagram for the data model, and a state diagram for how the game flows over time.

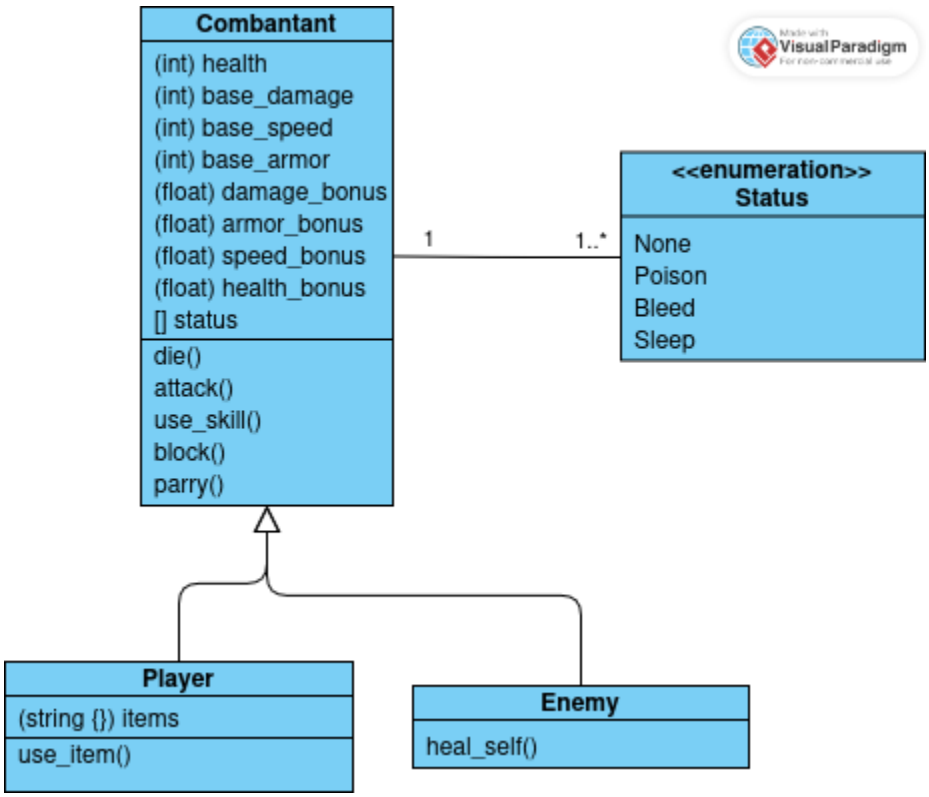
Diagram 1: Component diagram



The component diagram shows the major pieces and what each component depends on. At the bottom are the different scenes: MainMenu, MapScreen, CombatScene, VictoryScreen, DeathScreen, and HUD. The HUD is a reusable scene instanced inside CombatScene and MapScreen displaying health, relics, and other relevant information. Above the scenes on the right are the autoload singletons. GameState owns the current run, including the swordsman health, current map node, and the current game state. SceneManager listens for state changes and swaps the active scene. This way, no screen needs to know how to load each other.

The game's data lives in Data Resources, containing the MapData and EnemyData. MapData describes rooms/nodes and their connections, and EnemyData describes enemy stats. CombatScene reads EnemyData to build enemies, MapScreen reads MapData to draw the map (tree), and both read and write GameState. This separation allows us to change the UI while ensuring that said change can't break combat rules, and that data can change without editing scene logic. VictoryScreen and DefeatScreen are used to display relevant stats at the end of a run, whether that be beating the boss and winning (victory) or perishing before the former can be accomplished (defeat). MainMenu is used to house the start run and exit buttons. In the future we may add additional functionality that requires more menus in MainMenu, such as run history or loading previously saved games / runs.

Diagram 2: Class diagram

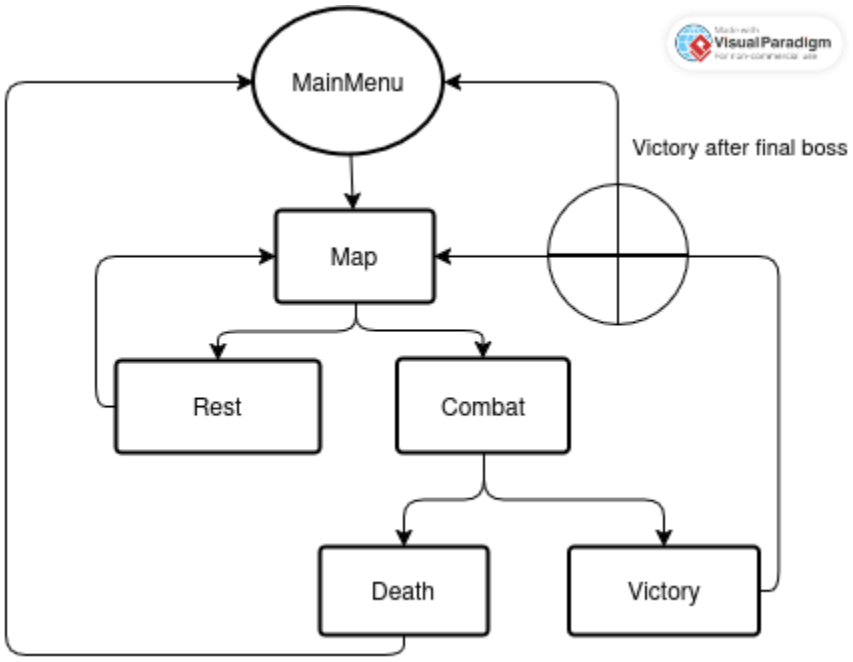


The class diagram shows the data model for fights. Combatant is one class that models both the player (swordsman) and enemies, so the game has a single definition of what it means to take part in combat. It stores core stats (health, base damage, base speed, and base armor) alongside separate floats for bonuses to the core stats. Keeping bonuses separate from base values lets relics, buffs / debuffs, and status effects modify a combatant without overwriting its original stats. Combatant also holds a list of active statuses and defines the actions every fighter shares: attack, block, parry, use skill, and die. Status is an enumeration of None, Poison, Bleed, and Sleep. Each combatant is associated with one or more Status values, with None used when

the combatant has no statuses affecting them. New effects can be added by extending the enumeration.

Player and Enemy extend Combatant and add only what is unique to them, so damage, death, and status handling are written once and apply equally to both sides. Player adds an item inventory and a use item action for healing and buff items. Enemy adds a heal self action which allows for enemies to heal themselves as part of their turn in combat. Enemy stats are loaded from data resource files rather than hardcoded so we can add new enemy types without having to change code.

Diagram 3: State diagram



Our third figure shows the behavior of the game over time as a state machine. There are six main states of the game, the first being MainMenu. Each state corresponds to a Godot scene and SceneManager, an autoload, handles the transitions. Launching the game lands the player at MainMenu, where runs can be started and the player can change settings (in the future). Starting the run resets GameState for a fresh run and moves to Map.

Map is the hub, how the player both reaches encounters and goes after winning; it emits the signal to switch nodes. SceneManager handles the signal by the node selected and from Map the player goes to either Combat or Rest depending on the node. In Rest, the player recovers before returning to Map to play on. Also from Map, the player enters Combat after moving to a fight node.

In Combat, the player and enemy take turns using actions like attacking or using skills such as block, parry. When one side runs out of health, the state is moved to Victory or Death based on which side lost/won. When the player dies, the Death screen is shown and the game moves to MainMenu. Alternatively, if the enemy dies, the Victory screen is shown before moving back to Map. However, if the enemy was the final boss the game returns to MainMenu instead.

Design decisions and future extensions

Data lives in Resources, behavior in Scenes, and run state in autoloads so each kind of change has its own separate home. Signals are used to keep scenes loosely coupled, allowing us to build the map and combat in parallel. Later in development and future additions can be plugged into this structure. For planned development, procedural generation of a tree map will replace the hardcoded map, skills, and status effects extend from the CombatScene, and relics are hooked directly into GameState to modify stats.

Sprint 1 Items (52 pts)

Combat

#14 Combat loop with win/loss, basic attack only (13)

When the swordsman engages with an enemy, the game state should change to "Combat." The enemy and swordsman should only have their basic attack type that deals a set amount of damage (example: -2hp). At the end of the battle, if the swordsman wins, there should be a screen that indicates "Victory" and the game state should change back to "Map." If the enemy wins then there should be a screen indicating "Defeated" and the game state change to "Main Menu."

#15 Enemies as data resources (5)

At least one basic enemy object should be implemented with (int) Health, (int) Base Damage, (int) Speed, (action) Action and (bool) Alive. If the enemy's health goes to 0, then (bool) Alive changes to False and the battle should end. An animation will play when the enemy does their basic attack. Animation frames should be provided in a sprite sheet found in the "assets" folder.

#16 Swordsman health in HUD (3)

When the game state is in "Combat," a static HUD should appear showing the swordsman's current below the swordsman. When the swordsman is attacked by the enemy, then the health bar should decrease. This should be implemented with the Godot Engine ProgressBar class (refer to documentation: https://docs.godotengine.org/en/stable/classes/class_progressbar.html).

#47 Combat UI: health bars, attack button, turn indicator (3)

A health bar should appear below the enemy and go down when the swordsman successfully attacks them. This should also be implemented with the ProgressBar class.

A small, rectangular menu should appear during the swordman's turn showing available actions for the player. For now, there should only be one choice showing "Basic attack." The CombatManager should start at PlayerTurn. After a single action from the player, the CombatManager should switch to EnemyTurn. Once both parties do at least one action each, (int) turnIndicator in the CombatManager should increment by 1.

Map

NEW: Hardcoded straight-line map (~3)

(description: start room, then easy enemy room, then hard enemy room)

Once the game changes from the MainMenu state to Map state, the swordsman should spawn in an empty room. The swordsman should be able to automatically progress to the second room containing a single BasicEnemy with (int) Health = 1 and display a "Victory" screen. Finally, the swordsman should advance to the final room which will be the same

BasicEnemy sprite but their (int) Base damage should one-shot the swordsman, displaying the "Defeated" screen. This should change the game state back to MainMenu.

#4 Map screen display (5)

During the Map state that occurs between MainMenu and the first encounter as well as between Combat states, there should be a full screen showing the map of all the rooms displayed in a tree-like manner.

#5 Map highlights, visited nodes, icons (3)

The Map screen should display icons indicating what is at the node. Enemy nodes should have a specific icon, shop nodes (do not implement, but are available in the "assets" folder), empty rooms etc. After the swordsman visits a node, the node icon should indicate that it was 'visited' (ex: change to a slightly darker color). While the player is choosing a node to advance to, when the cursor hovers over a node, it should be highlighted.

#6 Sprite travel along the map (3)

In the Map screen, there should be a sprite of the swordsman hovering over the current node he's at. When the player advances to a new node, an animation should play with the swordsman walking to the next node, stopping on the node icon, and fading the screen to black while the game state changes.

Menus and Flow

#53 Main menu (2)

MainMenu game state should display a screen with the title of the game (currently: Rogue Swordsman), button that says "Play" that when pressed will change the game state to "Map" and a button that says "Exit" that will exit the game and close the window.

#55 Death screen (3)

When the game state is "Death", the screen should display the text "Defeated," and a button that says "Return to Main Menu." When "Return to Main Menu" is pressed, the game state should change to "MainMenu."

#56 Victory screen (2)

When the game state is "Victory," the screen should display the text "Victory!," and a button that says "Go Onward." When "Go Onward" is pressed, the game state should change to "Map."

Infrastructure

#57 HTML5 export and public link (5)

Export the game and name the main file "index.html".

#59 Resizable window (2)

Set the base viewport to 640x480 and set the scaling to integer scaling to prevent pixel distortions. For now, keep a set aspect ratio by setting the "stretch aspect" to keep.

Refer to documentation/tutorial
(https://docs.godotengine.org/en/stable/tutorials/rendering/multiple_resolutions.html)